

▼

 DecodeLabs Internship – Project 1

Data Cleaning & Preparation

Objective

Clean and prepare the raw dataset by handling missing values, removing duplicate records, correcting data formats, and ensuring data integrity for accurate analysis.

▼

 Data Inspection

The dataset was explored to understand its structure, identify missing values, duplicate records, and data type inconsistencies.

```
import pandas as pd
import numpy as np
```

```
!ls

sample_data
```

```
from google.colab import files

uploaded = files.upload()
```

Choose files

Dataset for ...nalytics.xlsx

Dataset for Data Analytics.xlsx(application/vnd.openxmlformats-officedocument.spreadsheetml.sheet) - 125606 bytes, last modified: 26/06/2026 - 100% done

```
df = pd.read_excel("Dataset for Data Analytics.xlsx")
```

```
df.head()
```

	OrderID	Date	CustomerID	Product	Quantity	UnitPrice	ShippingAddress	PaymentMethod	OrderStatus	TrackingNumber	Item
0	ORD200000	2023-01-04	C72649	Monitor	5	570.62	928 Main St	Debit Card	Shipped	TRK37947903	
1	ORD200001	2024-08-23	C75739	Phone	2	151.35	823 Main St	Online	Shipped	TRK91186779	
2	ORD200002	2024-02-27	C81728	Tablet	5	550.68	512 Main St	Credit Card	Cancelled	TRK42903982	
3	ORD200003	2023-10-15	C33540	Chair	1	273.19	275 Main St	Debit Card	Returned	TRK62788070	
4	ORD200004	2025-05-08	C81840	Printer	4	626.01	668 Main St	Online	Delivered	TRK29241424	

Next steps: [Generate code with df](#) [New interactive sheet](#)

▼

Understand the structure of the dataset before cleaning.

```
print("Dataset Shape :", df.shape)
```

```
Dataset Shape : (1200, 14)
```

```
print(df.columns)

Index(['OrderID', 'Date', 'CustomerID', 'Product', 'Quantity', 'UnitPrice',
      'ShippingAddress', 'PaymentMethod', 'OrderStatus', 'TrackingNumber',
      'ItemsInCart', 'CouponCode', 'ReferralSource', 'TotalPrice'],
      dtype='object')
```

df.dtypes

Ø	
OrderID	object
Date	datetime64[ns]
CustomerID	object
Product	object
Quantity	int64
UnitPrice	float64
ShippingAddress	object
PaymentMethod	object
OrderStatus	object
TrackingNumber	object
ItemsInCart	int64
CouponCode	object
ReferralSource	object
TotalPrice	float64

dtype: object

df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1200 entries, 0 to 1199
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  -
0   OrderID                1200 non-null  object
1   Date                   1200 non-null  datetime64[ns]
2   CustomerID             1200 non-null  object
3   Product                1200 non-null  object
4   Quantity               1200 non-null  int64
5   UnitPrice              1200 non-null  float64
6   ShippingAddress        1200 non-null  object
7   PaymentMethod          1200 non-null  object
8   OrderStatus            1200 non-null  object
9   TrackingNumber         1200 non-null  object
10  ItemsInCart            1200 non-null  int64
11  CouponCode             891 non-null   object
12  ReferralSource         1200 non-null  object
13  TotalPrice             1200 non-null  float64
dtypes: datetime64[ns](1), float64(2), int64(2), object(9)
memory usage: 131.4+ KB
```

df.describe()

	Date	Quantity	UnitPrice	ItemsInCart	TotalPrice	
count	1200	1200.000000	1200.000000	1200.000000	1200.000000	
mean	2024-03-22 16:58:48	2.945833	356.412750	5.485000	1053.968300	
min	2023-01-01 00:00:00	1.000000	11.390000	1.000000	11.390000	
25%	2023-08-03 18:00:00	2.000000	186.062500	4.000000	410.520000	
50%	2024-03-23 00:00:00	3.000000	364.210000	5.000000	823.615000	
75%	2024-11-08 12:00:00	4.000000	521.570000	7.000000	1578.475000	
max	2025-06-30 00:00:00	5.000000	699.930000	10.000000	3456.400000	
std	NaN	1.407557	197.177146	2.281983	819.856558	

df.sample(10)

	OrderID	Date	CustomerID	Product	Quantity	UnitPrice	ShippingAddress	PaymentMethod	OrderStatus	TrackingNumber	I
993	ORD200993	2024-04-06	C61384	Monitor	3	267.45	576 Main St	Gift Card	Delivered	TRK33323210	
1183	ORD201183	2023-04-05	C12607	Desk	1	281.52	460 Main St	Gift Card	Shipped	TRK83653396	
565	ORD200565	2023-05-14	C62042	Tablet	5	307.98	214 Main St	Online	Pending	TRK42605492	
142	ORD200142	2023-09-06	C86374	Desk	5	272.77	573 Main St	Cash	Pending	TRK79451491	
676	ORD200676	2025-06-23	C90431	Laptop	2	422.34	985 Main St	Credit Card	Cancelled	TRK44151723	
33	ORD200033	2023-06-05	C52184	Chair	1	162.77	912 Main St	Online	Cancelled	TRK73312471	
474	ORD200474	2023-11-15	C72293	Printer	3	229.34	398 Main St	Credit Card	Shipped	TRK84378003	
953	ORD200953	2023-02-06	C43721	Phone	1	131.61	971 Main St	Cash	Shipped	TRK89392104	
1045	ORD201045	2024-09-20	C11893	Laptop	5	316.40	671 Main St	Gift Card	Returned	TRK72012758	
121	ORD200121	2023-11-30	C89822	Desk	1	410.78	673 Main St	Cash	Cancelled	TRK24138598	

✦ Identify missing values before cleaning the dataset.

```
df.isnull().sum()
```

	0
OrderID	0
Date	0
CustomerID	0
Product	0
Quantity	0
UnitPrice	0
ShippingAddress	0
PaymentMethod	0
OrderStatus	0
TrackingNumber	0
ItemsInCart	0
CouponCode	309
ReferralSource	0
TotalPrice	0

dtype: int64

```
missing_percentage = (df.isnull().sum()/len(df))*100  
  
missing_percentage
```

	0
OrderID	0.00
Date	0.00
CustomerID	0.00
Product	0.00
Quantity	0.00
UnitPrice	0.00
ShippingAddress	0.00
PaymentMethod	0.00
OrderStatus	0.00
TrackingNumber	0.00
ItemsInCart	0.00
CouponCode	25.75
ReferralSource	0.00
TotalPrice	0.00

dtype: float64

◆ Gemini

```
missing = df.isnull().sum()
missing[missing > 0]
```

	0
CouponCode	309

dtype: int64

✓ Replace missing values with suitable values while preserving data quality.

```
numerical_cols = ["Quantity", "UnitPrice", "ItemsInCart", "TotalPrice"]
```

```
for col in numerical_cols:
    df[col].fillna(df[col].median(), inplace=True)
```

/tmp/ipykernel_3858/518003605.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chain. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values is a copy.
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df

```
df[col].fillna(df[col].median(), inplace=True)
```

```
categorical_cols = [
    "Product",
    "ShippingAddress",
    "PaymentMethod",
    "OrderStatus",
    "TrackingNumber",
    "CouponCode",
    "ReferralSource",
    "CustomerID"
]
```

```
for col in categorical_cols:
    df[col].fillna(df[col].mode()[0], inplace=True)
```

/tmp/ipykernel_3858/295258120.py:13: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chain. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values is a copy.
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df

```
df[col].fillna(df[col].mode()[0], inplace=True)
```

```
df.isnull().sum()
```

```

      0
OrderID    0
Date       0
CustomerID 0
Product    0
Quantity   0
UnitPrice  0
ShippingAddress 0
PaymentMethod 0
OrderStatus 0
TrackingNumber 0
ItemsInCart 0
CouponCode 0
ReferralSource 0
TotalPrice 0

```

```
dtype: int64
```

```

duplicates=df.duplicated().sum()
duplicates

```

```
np.int64(0)
```

```
print("Duplicate records :", duplicates)
```

```
Duplicate records : 0
```

```
df[df.duplicated()]
```

```

OrderID  Date  CustomerID  Product  Quantity  UnitPrice  ShippingAddress  PaymentMethod  OrderStatus  TrackingNumber  ItemsInCa

```

Remove Duplicate Records

```
df.drop_duplicates(inplace=True)
```

```
print(df.duplicated().sum())
```

```
0
```

Check Duplicate Order IDs

```
df["OrderID"].duplicated().sum()
```

```
np.int64(0)
```

Standardize Date Format

```
df["Date"] = pd.to_datetime(df["Date"])
```

```
df["Date"] = pd.to_datetime(df["Date"])
```

```
df["Date"] = df["Date"].dt.strftime("%Y-%m-%d")
```

Standardize Text Columns

```
text_columns = [
    "Product",
    "ShippingAddress",
    "PaymentMethod",
    "OrderStatus",
    "CouponCode",
    "ReferralSource",
    "CustomerID",
    "TrackingNumber"
]

for col in text_columns:
    df[col] = df[col].astype(str).str.strip().str.title()
```

Verify Data Types

```
df.dtypes
```

	0
OrderID	object
Date	object
CustomerID	object
Product	object
Quantity	int64
UnitPrice	float64
ShippingAddress	object
PaymentMethod	object
OrderStatus	object
TrackingNumber	object
ItemsInCart	int64
CouponCode	object
ReferralSource	object
TotalPrice	float64

```
dtype: object
```

Final Validation

```
print("Missing Values")
print(df.isnull().sum())

print("\nDuplicate Records")
print(df.duplicated().sum())

print("\nDataset Shape")
print(df.shape)

print("\nData Types")
print(df.dtypes)
```

```
Missing Values
OrderID      0
Date         0
CustomerID   0
```

```
Product      0
Quantity     0
UnitPrice    0
ShippingAddress 0
PaymentMethod 0
OrderStatus  0
TrackingNumber 0
ItemsInCart  0
CouponCode   0
ReferralSource 0
TotalPrice   0
dtype: int64

Duplicate Records
0

Dataset Shape
(1200, 14)

Data Types
OrderID      object
Date         object
CustomerID   object
Product      object
Quantity     int64
UnitPrice    float64
ShippingAddress object
PaymentMethod object
OrderStatus  object
TrackingNumber object
ItemsInCart  int64
CouponCode   object
ReferralSource object
TotalPrice   float64
dtype: object
```

Export Clean Dataset

```
df.to_excel("Cleaned_Dataset.xlsx", index=False)
```

Project Summary

```
print("="*60)
print("      DATA CLEANING PROJECT SUMMARY")
print("="*60)

print(f"Total Rows      : {df.shape[0]}")
print(f"Total Columns    : {df.shape[1]}")
print(f"Missing Values     : {df.isnull().sum().sum()}")
print(f"Duplicate Records  : {df.duplicated().sum()}")
print("Date Format      : Standardized")
print("Text Formatting   : Standardized")
print("Dataset Status    : Ready for Analysis")

print("="*60)
```

```
=====
      DATA CLEANING PROJECT SUMMARY
=====
Total Rows      : 1200
Total Columns   : 14
Missing Values  : 0
Duplicate Records : 0
Date Format     : Standardized
Text Formatting : Standardized
Dataset Status  : Ready for Analysis
=====
```

 Data Cleaning Completed

The dataset has been successfully cleaned and validated. It is now consistent, accurate, and ready for further analysis and visualization.

✓ Top 5 selling products

```
df["Product"].value_counts().head(5)
```

	count
Product	
Printer	181
Tablet	179
Chair	178
Laptop	173
Desk	170

dtype: int64

✓ Total Revenue

```
df['TotalPrice'].sum()
```

```
np.float64(1264761.96)
```

✓ Order status count

```
df['OrderStatus'].value_counts()
```

	count
OrderStatus	
Cancelled	250
Returned	247
Pending	237
Shipped	235
Delivered	231

dtype: int64

```
df.to_excel("Cleaned_Dataset.xlsx", index=False)
```

```
print("Dataset Saved Successfully")
```

```
Dataset Saved Successfully
```

```
from google.colab import files
files.download("Cleaned_Dataset.xlsx")
```

✓ Final Conclusion

Objective

Summarize the completed data cleaning process.

```
print("="*60)
print("🎉 PROJECT COMPLETED SUCCESSFULLY")
print("="*60)

print("""
✓ Missing values handled
✓ Duplicate records removed
```



```
✓ Date format standardized
✓ Text values cleaned
✓ Dataset validated
✓ Dataset exported successfully
```

```
Status:
READY FOR DATA ANALYSIS
"""
```

```
print("="*60)
```

```
=====
🎉 PROJECT COMPLETED SUCCESSFULLY
=====
```

```
✓ Missing values handled
✓ Duplicate records removed
✓ Date format standardized
✓ Text values cleaned
✓ Dataset validated
✓ Dataset exported successfully
```

```
Status:
READY FOR DATA ANALYSIS
```

```
=====
```

Conclusion

The raw dataset was transformed into a clean and structured dataset by resolving missing values, removing duplicates, correcting formats, and validating data quality. The final dataset is analysis-ready and suitable for visualization and predictive analytics.